

Full-Stack Project – Standard Requirements

Frontend: HTML, CSS, JavaScript | Backend: Node.js + Express

1. UI / Design

- Create a clean, modern, and organized user interface.
- Use a consistent color palette, typography, spacing, buttons, and form styles.
- Keep the design simple and easy to understand; avoid unnecessary elements.
- All pages should have a consistent visual identity.
- Use suitable images/icons when they improve the user experience.

2. Responsive Design

- The website must work correctly on desktop, tablet, and mobile screens.
- Do not allow horizontal scrolling on normal screen sizes.
- Navigation, cards, forms, images, tables, and other components must adapt to smaller screens.
- Test the project using different screen sizes before submission.

3. Complete User Flow

- The website should have a clear story/flow for the user from entering the site until completing the main action.
- The user should be able to navigate between the main pages easily.
- The project should provide meaningful pages based on its idea (for example: Home, About, Products/Services, Details, Contact, Dashboard, etc.).
- Do not build isolated pages; the pages should work together as one complete website.

4. Frontend Requirements

- Use pure HTML, CSS, and JavaScript. No frontend framework is required.
- Use semantic HTML elements where appropriate.
- Use JavaScript to make the interface interactive.
- Validate important user inputs on the frontend.
- Show clear feedback for actions such as successful operations, errors, loading, empty results, and invalid input.
- Organize frontend files clearly (HTML, CSS, JS, images/assets, etc.).

5. Backend Requirements

- Use Node.js and Express.js.
- Create a clear backend structure (routes, controllers, models/services when needed, middleware, configuration, etc.).
- Create RESTful API endpoints suitable for the project.
- Use proper HTTP methods such as GET, POST, PUT/PATCH, and DELETE when needed.
- Handle errors properly and return clear HTTP status codes.
- Do not put all backend logic inside one file.

6. Database & CRUD

- The project should use a database when the idea requires stored data.

- Implement CRUD operations where appropriate: Create, Read, Update, and Delete.
- Data should be connected to the real application flow, not just created as unused APIs.
- Handle cases such as missing data, invalid IDs, duplicate data, and empty results.

7. Authentication & Authorization

- If the project includes users, implement a proper registration and login flow.
- Passwords must not be stored as plain text.
- Protect private routes/endpoints when necessary.
- Different user roles can be added when the project idea needs them (for example: Admin and User).
- Users should only be able to access actions/data allowed for their role.

8. Frontend ↔ Backend Integration

- The frontend must communicate with the Express API using JavaScript (for example, fetch).
- Display real backend/database data in the UI.
- User actions such as submitting forms, adding items, editing data, or deleting data should be connected to the backend when applicable.
- Do not submit a project where the frontend and backend are working separately without real integration.

9. Forms & Validation

- Create user-friendly forms with clear labels and inputs.
- Validate required fields and basic data formats.
- Show useful error messages instead of generic messages.
- Handle both frontend validation and backend validation.
- .

10. Git & Teamwork

- Use Git and GitHub for the project.
- Each team member should contribute through Git with meaningful commits.
- Use clear commit messages.
- Keep the repository organized and include a README file.
- Do not upload unnecessary folders or sensitive files such as node_modules or .env.

11. Final Submission Checklist

- The website is fully responsive.
- All main pages and user flows are connected.
- Frontend and backend are properly integrated.
- Main CRUD operations work correctly where applicable.
- Forms and validation work correctly.
- Authentication/authorization works if required by the project.
- API endpoints work correctly and handle errors.
- No major console errors or broken links.
- Code is organized and readable.
- GitHub repository is clean and contains a clear README.
- Project is tested on both desktop and mobile.